

An Open Book: Level 4 Order Book Data from the Hyperliquid Exchange

Jakob Albers^{a,*}, Mihai Cucuringu^{b,c}, Sam Howison^d, Alexander Y. Shestopaloff^{e,f}

^a*Department of Statistics, University of Oxford, Oxford, UK*

^b*Department of Mathematics, University of California Los Angeles, CA, USA*

^c*Oxford-Man Institute of Quantitative Finance, University of Oxford, Oxford, UK*

^d*Mathematical Institute, University of Oxford, Oxford, UK*

^e*School of Mathematical Sciences, Queen Mary University of London, London, UK*

^f*Department of Mathematics and Statistics, University of Guelph, ON, Canada*

Abstract

This paper presents a high-frequency cryptocurrency market dataset obtained by operating a non-validating blockchain node on Hyperliquid, a limit order book-based decentralised exchange that is now one of the most liquid cryptocurrency exchanges, handling billions of USD in daily turnover. Unlike conventional Level 3 data sources such as LOBSTER, our data captures the complete order placement and cancellation history of all traders—identified pseudonymously through wallet addresses—including rejected orders and failed cancellation attempts that typically do not appear in conventional feeds, counterparty identities on both sides of each trade, and inventory positions after every transaction, all with nanosecond-precision timestamps. We make a portion of this dataset publicly available on Zenodo (with more available upon request), and discuss potential applications to market microstructure research.

1. Introduction

Researchers studying market microstructure in general, and limit order books (LOBs) in particular, currently have limited options for obtaining detailed, high-quality datasets. Most commonly, existing research utilises databases such as LOBSTER [5], which, although valuable, offer an incomplete record of market activity and lack trader identifiers. Alternatively, researchers may negotiate access to proprietary datasets directly from exchanges or regulators—arrangements that are difficult to secure and that typically cannot be shared with other researchers wishing to interrogate the original findings. Commercial market data feeds, while more accessible, are generally not designed with academic research needs in mind and lack the granularity required for many investigations of interest.

In this paper we present a dataset of LOB activity obtained from Hyperliquid, a decentralised cryptocurrency exchange operating entirely on a blockchain. In just a short period, Hyperliquid has grown into one of the most liquid cryptocurrency exchanges, handling billions of USD in transaction turnover every day. Hyperliquid records all transactions publicly on a blockchain ledger, making them, in principle, accessible to anyone, much as for other blockchains such as Bitcoin. However, the raw transaction stream is large—upwards of 100 GB per day—and converting it into a form

*Corresponding author. Email: jakob.albers@merton.ox.ac.uk

suitable for quantitative analysis requires infrastructure for handling large datasets. We have set up a non-validating node on the Hyperliquid blockchain and developed a data processing pipeline to transform the raw transaction stream into a structured, compact dataset suitable for research. We make a portion of this dataset publicly available on Zenodo, along with documentation and code for working with the data, to facilitate further research. The resulting dataset provides what Hyperliquid terms “Level 4” data¹—extending the conventional Level 3 granularity with pseudonymous trader identifiers, rejected order/cancellation attempts, and counterparty information for every trade.

1.1. Comparison with Conventional Data

As noted above, existing datasets such as LOBSTER typically only capture a subset of the overall order book activity, and even within that subset, they provide incomplete information in a way which rules out certain investigations of interest. In contrast, our Hyperliquid dataset offers:

- **User information attached to each action:** Every action is tagged to a user wallet address.
- **Full order information:** Unlike LOBSTER, where the order type for a trade (such as limit or market order) is not directly visible, Hyperliquid records all order details, including order type, time-in-force,² limit price, size, and all other parameters for each submitted order.
- **Failed attempts:** We observe not only successful requests but also all failed order attempts (such as rejected maker orders). The data also captures every cancellation request, including repeated or unsuccessful attempts.
- **Counterparty identification:** Each transaction contains the identifiers of both the maker and the taker counterparties of the trade, along with their starting positions before the trade.
- **User-level statistics:** From the trade and order data, user-level metrics such as position sizes, realised profit and loss, and inventory trajectories can be computed for any trader over any time horizon.

2. Hyperliquid Exchange

Hyperliquid is a decentralised exchange (DEX) built on its own Layer 1 blockchain—a distributed ledger where transactions are grouped into blocks, cryptographically linked in sequence, and validated by a network of nodes through a consensus mechanism.³ Hyperliquid employs a purpose-built,

¹Hyperliquid distinguishes Level 2 (aggregated depth, available via the public API) from Level 4 (individual orders with wallet addresses, order IDs, and full parameters, available through a node). In the standard market data taxonomy, Level 3 denotes individual order events (e.g. LOBSTER). Level 4 extends this with pseudonymous trader identifiers, rejected order attempts, counterparty information, and pre-trade inventories. See Hyperliquid’s node documentation: <https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/nodes/l1-data-schemas>.

²A directive that specifies the conditions under which an order remains active and how it may interact with the book (e.g. post-only, immediate-or-cancel, good-til-cancel).

³Some have questioned Hyperliquid’s degree of decentralisation: the active validator set comprises only 21 nodes, selected as the operators with the highest staked holdings of the platform’s native token (HYPE). This contrasts with more permissionless designs such as Bitcoin, where thousands of nodes participate in consensus.

closed-source consensus protocol (HyperBFT) that produces blocks approximately every 100–200 ms—significantly faster than most other blockchains—enabling throughput sufficient for high-frequency trading strategies (Hyperliquid claims a capacity of up to 200,000 orders per second).⁴

Unlike the majority of DEXs—which use so-called “automated market maker” (AMM) designs (see e.g. [6] and [3])—Hyperliquid operates a fully on-chain LOB, much like those on centralized exchanges within crypto and other asset classes. Order matching follows standard price-time priority. We assume familiarity with LOBs; readers seeking an introduction are referred to [4] or other standard literature.

Unlike traditional exchanges, Hyperliquid requires no account creation or identity verification for traders. Users simply connect an Ethereum wallet⁵ to the platform and deposit USDC (the sole accepted collateral) from the Arbitrum L2 network, with maximum leverage up to 50× depending on the instrument.

Order Types and Trading Parameters. Hyperliquid provides a suite of order types and parameters, comparable to those on major centralized cryptocurrency exchanges such as Binance or Bybit, and similar to those available on traditional venues such as NASDAQ or Euronext. The complete list is as follows:

- **Limit Order:** Submits an order at a specified price and quantity; rests on the book if not marketable.
- **Market Order:** Executes immediately against the best available liquidity.
- **Stop Market:** A market order that activates when a trigger price is reached, typically used to limit losses.
- **Stop Limit:** A limit order that activates when a trigger price is reached.
- **Scale Order:** Places multiple limit orders across a specified price range.
- **TWAP (Time-Weighted Average Price) Order:** An algorithmic order that slices a large trade into smaller sub-orders executed at regular intervals.

Additionally, these orders can be modified with the following time-in-force and conditional parameters:

- **Post-Only (ALO - Add Liquidity Only):** Ensures the order can only provide liquidity; it is cancelled if it would cross the spread and take liquidity.
- **Immediate-Or-Cancel (IOC):** Fills any available portion of the order immediately against resting liquidity and cancels any unfilled remainder.

⁴On traditional equity venues, trade execution and settlement are often separated: matched trades are forwarded to a central counterparty such as the DTCC, with final settlement occurring on a T+1 cycle (T+2 in many non-US markets). On Hyperliquid, matching and settlement are handled by the same on-chain state machine; positions and margin update atomically within each block, with no clearing intermediary or settlement delay.

⁵Unlike on centralised exchanges, where traders deposit funds into accounts controlled by the exchange operator, Hyperliquid users retain custody of their assets through their own cryptographic keys. The exchange can execute trades against the deposited margin but cannot unilaterally move or misappropriate user funds.

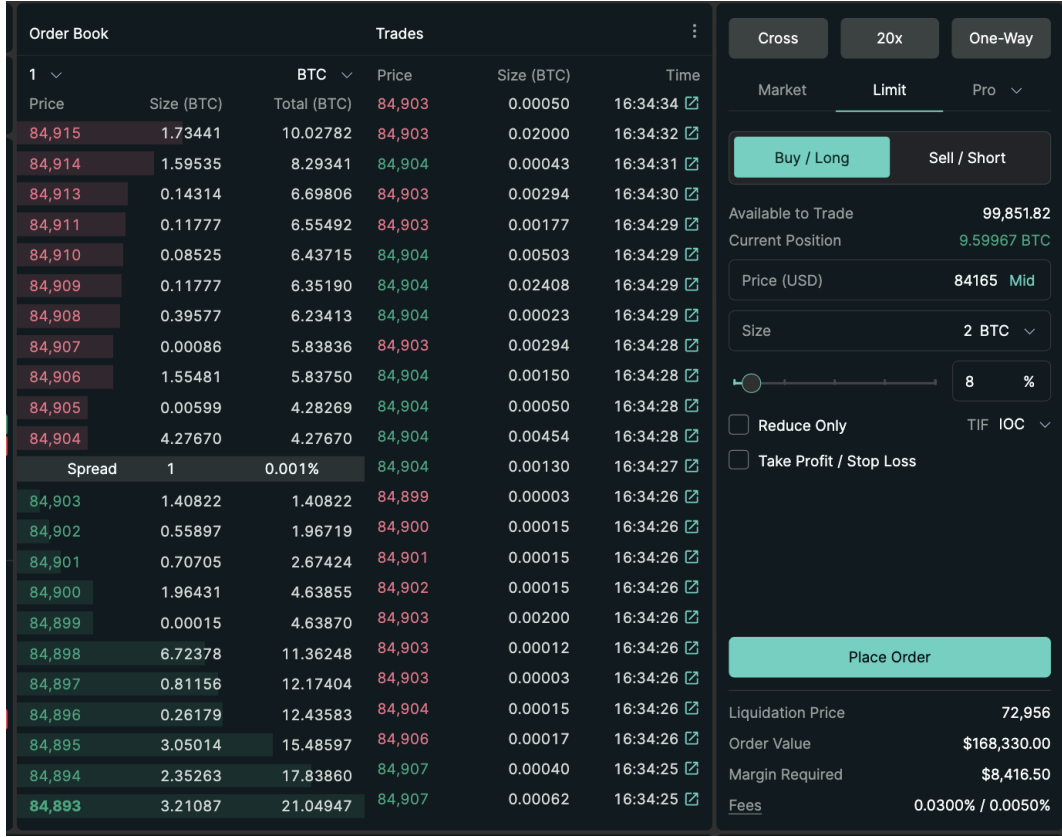


Figure 1: Hyperliquid's trading interface for the BTC perpetual. Left: The order book displaying price levels with corresponding quantities and cumulative quantities. Center: Time-sequenced transaction log showing recent trades. Right: Order placement panel with multiple order types (Limit, Market, Stop, TWAP) and time-in-force options.

- **Good-Til-Cancel (GTC):** The default behaviour where an order rests on the book until it is either fully filled or explicitly cancelled.
- **Reduce-Only:** Ensures the order can only reduce an open position, not increase it or open a new position in the opposite direction.
- **Take Profit (TP) and Stop Loss (SL):** Triggered market orders used to close a position to realize profits or limit losses, respectively.

Fee Structure and Market Access. Hyperliquid employs a standard fee schedule with maker-taker fees. Makers pay lower fees than takers, with a baseline of 1.5 basis points (bp) for makers and 4.5 bp for takers. A VIP tier system offers progressively lower fees based on trading volume within each individual wallet address. Traders or entities may create and use an unlimited number of wallet addresses (user IDs); however, fee tiers are determined independently for each wallet. Therefore, concentrating trading activity into fewer (or a single) wallets typically results in overall lower fees, compared to spreading volume across multiple wallets with correspondingly higher fees. The most active traders enjoy rates as low as -0.3 bp (earning a rebate) for maker orders and 1.44 bp for taker orders. Separately from the maker-taker schedule, all order submissions and

cancellations on Hyperliquid are gasless. Most blockchain transactions—sending Bitcoin or Ethereum between addresses, interacting with smart contracts, trading on other decentralised exchanges—incur a per-transaction network fee (commonly termed “gas”). Such a fee in a LOB market would make high-frequency strategies prohibitively expensive, considering, for example, the high cancel-to-trade ratios characteristic of many such strategies; conversely, the absence of this cost on Hyperliquid ensures that high-frequency trading remains structurally viable.

Market participants can interact with Hyperliquid through its web interface or programmatically via its public API. The API provides WebSocket streams for real-time market data (such as executed trades and order book updates) and REST endpoints for private order management, facilitating algorithmic trading strategies and bot development. However, these publicly available WebSocket streams do not provide a full message-level feed nor include user identifiers (wallet addresses). Thus, they lack the granularity available by running a node and parsing every blockchain transaction, which captures *all* submitted actions in the market—including failed, cancelled, and partially filled orders—alongside user and action identifiers.

Order Lifecycle. Every order action on Hyperliquid—submission, cancellation, modification, execution, and rejection—generates a discrete on-chain event, timestamped to nanosecond precision and permanently linked to the originating wallet address and order identifier. The complete chain of events for any order can therefore be reconstructed from the data.

Intra-Block Action Ordering. Within each block (produced approximately every 100-200 ms), Hyperliquid imposes a type-based priority hierarchy: passive orders are processed first, cancellations second, and taker orders last; within each tier, arrival-time priority applies. The rationale is maker protection—with cancellations processed before aggressive orders in the same block, market makers are more likely to be able to cancel stale quotes before being adversely selected, encouraging liquidity provision.

A distinct but related feature of Hyperliquid’s distributed nature is that arrival time itself is not uniquely defined. On a centralised exchange, a single matching engine imposes a definitive ordering on all incoming messages. On Hyperliquid, messages are routed through geographically distributed validators, one of which is selected pseudorandomly to propose each block; because different validators may sequence the same pending messages differently, the within-block ordering reflects the proposer’s local view rather than a globally consistent ordering.

The practical significance of intra-block hierarchy is, however, constrained by the unpredictability of block assignment. Traders cannot control or reliably predict which block a given message will land in; two messages sent simultaneously may arrive several blocks apart. A cancellation submitted milliseconds *after* an incoming aggressive order may land in an *earlier* block, causing the cancellation to execute before the aggressive order even exists on the book. Conversely, a cancellation intended to pull a stale quote may arrive too late. The extent to which any participant can exploit the intra-block type hierarchy is therefore bounded by the substantial randomness in message routing.

Further Resources. Readers interested in exploring Hyperliquid’s markets may find the following resources useful:

- **Live BTC Order Book:** Real-time view of the BTC perpetual futures order book and trading interface (see Figure 1).
- **Trader Leaderboard:** Rankings of top traders on Hyperliquid.
- **Exchange Statistics:** Aggregate platform metrics such as number of users, trading volumes, open interest, and much more.

- **Hypurrscan Blockchain Explorer:** Allows tracking of any user's trading activity by pasting a wallet address in the search field, providing full order and trade history along with position details and the user's PnL.
- **Community Discord:** Active community server with channels for trading discussion, technical questions, and developer support.

3. Data

3.1. Node Deployment and Data Collection

Our data were collected by operating a non-validating node on the Hyperliquid blockchain. In blockchain terminology, a *validating* node participates in the consensus mechanism that determines which transactions are accepted into the ledger; a *non-validating* node synchronises with the network and maintains a complete local copy of every transaction without participating in consensus. Because Hyperliquid records every action in its limit order book—submissions, cancellations, executions, and rejections across all listed instruments—as a blockchain transaction, a non-validating node captures the complete history of the market's activity.

Deploying such a node requires no special approval, financial commitment, or blockchain development experience. One provisions a Linux server (Ubuntu 22.04 or 24.04 LTS) meeting the recommended specification of 16 vCPUs, 64 GB RAM, and a 500 GB SSD from any cloud provider (e.g. AWS, Google Cloud, DigitalOcean), downloads the node binary from Hyperliquid's repository,⁶ and starts it with logging flags that control which data streams are written to disk. The complete deployment is shown below:

Listing 1: Deployment of a Hyperliquid non-validating node with order status and trade logging enabled.

```
# Download the node binary
curl https://binaries.hyperliquid.xyz/Mainnet/hl-visor \
  -o ~/hl-visor && chmod a+x ~/hl-visor

# Configure the node to connect to the live network
echo '{"chain": "Mainnet"}' > ~/visor.json

# Start the node with data-logging flags (in tmux or screen)
~/hl-visor run-non-validator \
  --write-order-statuses \
  --write-trades \
  --write-raw-book-diffs
```

The flag `-write-order-statuses` records every order event (placements, cancellations, fills, and rejections); `-write-trades` records every executed trade with counterparty details; and `-write-raw-book-diffs` records every change to the visible limit order book.⁷ On first launch the node replays the chain

⁶Full setup instructions and the node binary are maintained at <https://github.com/hyperliquid-dex/node>. The data schemas produced by the node are documented at <https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/nodes/l1-data-schemas>.

⁷Additional flags are available, including `-write-misc-events` (deposits, withdrawals, liquidations, and other ancillary events); see the official documentation for a full list.

history to reach the current block height—a process that typically takes several hours—after which it writes data to disk in real time. Because the node can generate on the order of 100 GB of raw logs per day when all data flags are enabled, we recommend attaching at least 2 TB of additional storage for extended collection periods. During December 2025, the node recorded approximately 880 million order status events per day for the three coins in our sample, with daily volumes ranging from roughly 730 million on quieter days to over 1.1 billion during periods of elevated activity. Trade records are orders of magnitude fewer, as each trade corresponds to a single fill event rather than the many order submissions, cancellations, and rejections that surround it.

3.2. Raw Data Format

We now describe the fields in each of the three data streams introduced above.

3.2.1. Order Status Records

The order status stream records every event for every order submitted to the matching engine. Each record represents a single event in an order’s history: acceptance onto the book (**open**), full execution (**filled**), cancellation (**canceled**), or rejection (the most common being **badAloPxRejected**, when an Add-Liquidity-Only order would cross the spread; see Albers et al. [1, 2]). Partial fills generate one or more **filled** events with decreasing remaining size, optionally followed by a **canceled** event for the unfilled remainder. Table 1 lists every field, illustrated with a representative example; full technical details are provided in `SCHEMA.md`.

Table 1: All fields in each order status record as received from the node (JSON), illustrated with a rejected post-only (ALO) sell order on BTC.

Field	Description	Example
<i>Top-level fields</i>		
time	Event timestamp (nanosecond precision)	2026-02-18T22:59:59.818527771
user	Trader's wallet address (pseudonymous identifier)	0xd86a...f00e
status	Event outcome: open , filled , canceled , badAloPxRejected , among 14 others	badAloPxRejected
hash	L1 transaction hash; null if in the same block as the order submission	null
builder	Builder/vault routing info (object if routed through a builder, null otherwise)	null
<i>Nested order object</i>		
coin	Instrument symbol	BTC
side	Order side: A = sell/ask, B = buy/bid	A
limitPx	Limit price (decimal string)	66364.0
origSz	Original order size at submission (decimal string)	0.19863
sz	Remaining order size at time of this event	0.19863
oid	Unique order identifier; links all lifecycle events for the same order	324082134013
timestamp	Original submission time (milliseconds since epoch)	1771455599818
orderType	Order type: Limit, Market, Stop Market, Stop Limit, TP Market, TP Limit	Limit
tif	Time-in-force: Alo (post-only), Gtc , Ioc , or null (for trigger orders)	Alo
reduceOnly	Whether the order can only reduce an existing position	false
cloid	Client-supplied order identifier (hex string), or null	0x6355...3900
<i>Trigger/conditional fields (within order)</i>		
isTrigger	Whether this is a conditional/trigger order	false
triggerPx	Trigger activation price (0.0 if not a trigger order)	0.0
triggerCondition	Trigger condition as a string (e.g. " Price below 3957.8 ", or " N/A ")	N/A
children	Array of linked child TP/SL orders (empty if none)	[]
isPositionTpsl	Whether this is a TP/SL applied to the trader's entire position (as opposed to a single order)	false

3.2.2. Trade Records

The trade stream records every executed trade across all listed perpetual contracts. Each record contains the standard fields—price, size, timestamp, and aggressor side—but goes substantially further: both counterparties are identified by wallet address, along with their respective pre-trade inventory positions and order identifiers. Those order identifiers allow trades to be linked back to the order status records described in Section 3.2.1, and, for TWAP executions (see Section 2), a shared `twap_id` further links child fills to their parent meta-order. Which of the two counterparties

is the maker and which the taker is determined as follows: when `side = A`, `side_info[0]` is the maker (buyer) and `side_info[1]` the taker (seller); the roles reverse when `side = B`. Table 2 lists every field, illustrated with a complete example including both counterparties.

Table 2: All fields in each trade record as received from the node (JSON), illustrated with a sell-aggressor trade on BTC. Each trade contains exactly two counterparties in the `side_info` array.

Field	Description	Example
<i>Trade-level fields</i>		
<code>coin</code>	Instrument symbol	BTC
<code>side</code>	Aggressor side: A = sell (taker hit the bid), B = buy (taker lifted the offer)	A
<code>time</code>	Trade timestamp (nanosecond precision, ISO 8601)	2026-02-19T08:00:00.660094422
<code>px</code>	Fill price (decimal string)	67189.0
<code>sz</code>	Fill size in base asset units (decimal string)	0.00018
<code>hash</code>	L1 transaction hash (0x000...0 if in the same block as the order submission)	0xa939...49ef
<code>trade_dir_override</code>	Direction override indicator (usually Na)	Na
<i>Counterparty 1 (side_info[0])</i>		
<code>user</code>	Trader's wallet address	0x220c...1439
<code>start_pos</code>	Position size <i>before</i> the trade (positive = long, negative = short)	0.0
<code>oid</code>	Order identifier that generated this fill	324311341698
<code>twap_id</code>	TWAP order identifier; <code>null</code> if not a TWAP execution	null
<code>cloid</code>	Client-supplied order identifier; <code>null</code> if not provided	null
<i>Counterparty 2 (side_info[1])</i>		
<code>user</code>	Trader's wallet address	0xc6ac...0e84
<code>start_pos</code>	Position size before the trade	-56.15982
<code>oid</code>	Order identifier that generated this fill	324311367793
<code>twap_id</code>	TWAP order identifier; <code>null</code> if not a TWAP execution	null
<code>cloid</code>	Client-supplied order identifier; <code>null</code> if not provided	0x0000...9b18

3.2.3. Raw Book Diff Records

The book diff stream records every change to the visible limit order book. Each record identifies the user (by wallet address), the order ID, coin, side, price level, and the nature of the change: a new order placed on the book ("**new**", with a size field), an order removed ("**remove**", from cancellation or full fill), or a size update after a partial fill ("**update**", with original and remaining size).

Unlike the order status stream, which captures every order *attempt*—including the approximately 89% that are rejected by the matching engine—the book diff stream records only orders that were actually accepted onto the book and their subsequent modifications. The `oid` field links book diff records to the corresponding order status records, enabling researchers to join the two streams. Full field-level documentation is provided in `SCHEMA.md`.

3.3. Compression and Binary Encoding

In its native JSON format, a single order status record occupies approximately 300–450 bytes depending on field content; at roughly 880 million events per day, this implies an uncompressed daily total size exceeding 300 GB—well beyond what most analytical workflows or academic repositories can accommodate. For the public release, we therefore convert the raw order status logs into a compact binary encoding⁸ of 54 bytes per record, achieving an 85–88% reduction in file size while preserving all analytically relevant fields (the transaction hash and client order ID are omitted; see `SCHEMA.md` for the complete binary field list). This compression was motivated in part by Zenodo’s 200 GB upload limit, but the more compact representation also allows downstream users to load and process the data substantially faster than would be possible with the original JSON.

The encoding applies three principal transformations. First, wallet addresses (42-character hexadecimal strings) are mapped to integer user identifiers via a lookup table (`users.csv`). Second, categorical fields—order status, order type, and time-in-force—are encoded as compact unsigned integer codes via separate lookup tables (`statuses.csv`, `order_types.csv`, and `tifs.csv`). Third, prices and sizes are packed into fixed-width integers (32-bit) using a custom fixed-point scheme. As an illustration, Table 3 shows how selected fields from the raw JSON record in Table 1 map to their binary-encoded counterparts.

Table 3: Illustration of the binary encoding for selected fields. The raw JSON field (left) is transformed into a compact binary representation (right) via lookup tables and fixed-point arithmetic.

Raw JSON field	Raw value	Binary field	Bytes
<code>user</code>	0xd86a...f00e (42 chars)	<code>userId (uint32)</code>	4
<code>side</code>	A	<code>isAsk (bool)</code>	1
<code>limitPx</code>	66364.0	<code>uint32 (fixed-point)</code>	4
<code>status</code>	badAloPxRejected (16 chars)	<code>statusId (uint8)</code>	1
<code>time</code>	2026-02-18T22:59:59...	<code>ts (uint64, ns)</code>	8
<code>coin</code>	BTC	(implicit from filename)	—
...	...	...	...
Total	~300–450 bytes	Packed binary record	54

The full binary layout and all decoding procedures are documented in `SCHEMA.md`. The trade data, which are already sufficiently compact, are retained in their original gzip-compressed JSON format without binary encoding.

Reader Code. A standalone Python script (`read_data.py`) is included with the release. It decodes the binary order status files into pandas DataFrames and loads the JSON trade and book diff records through a unified interface, allowing a researcher to begin working with the data within minutes of download.

⁸A binary encoding stores data as fixed-width numerical values in raw bytes, rather than the variable-length text strings used by JSON. This eliminates the overhead of field names, quotation marks, and delimiters, and allows downstream tools to read records at fixed byte offsets without parsing.

4. Data Availability

The dataset accompanying this paper is publicly available on Zenodo under a Creative Commons Attribution 4.0 International (CC BY 4.0) licence.⁹ It comprises three components: (i) order status records for December 2025, covering the three most liquid perpetual contracts (BTC, ETH, SOL), stored in the binary encoding described in Section 3.3 and split into six archives by coin and order outcome (accepted versus rejected, with the latter comprising approximately 89% of all records—to our knowledge, no other public LOB dataset records failed order attempts); (ii) raw book diff records for December 2025, capturing every change to the visible limit order book for the same three contracts in gzip-compressed JSON format, packaged as a single archive; and (iii) trade execution data for all perpetual contracts from October 2025 through January 2026, in gzip-compressed JSON format, packaged into four monthly archives (a small number of gaps due to infrastructure restarts are documented in `README.md`). December 2025 order status and book diff coverage is complete with no gaps. Table 4 summarises all released archives.

Table 4: Contents of the publicly released dataset.

Component	Archive	Size
Order statuses (Dec 2025)	<code>btc_orders_202512.tar.xz</code>	19 GB
	<code>btc_rejected_202512.tar.xz</code>	46 GB
	<code>eth_orders_202512.tar.xz</code>	12 GB
	<code>eth_rejected_202512.tar.xz</code>	24 GB
	<code>sol_orders_202512.tar.xz</code>	6.3 GB
	<code>sol_rejected_202512.tar.xz</code>	8.5 GB
Raw book diffs (Dec 2025)	<code>book_diffs_202512.tar</code>	50 GB
Trades (Oct 2025–Jan 2026)	4 monthly <code>.tar</code> archives	30 GB
Lookup tables	<code>mapdir.tar.xz</code>	10 MB
Documentation	<code>SCHEMA.md</code> , <code>read_data.py</code> , <code>README.md</code>	<1 MB

The total download size is approximately 196 GB. Together, the order status and book diff streams provide a complete picture of market activity: the former captures every order *attempt* (including rejected orders that never reach the book), while the latter records every resulting change to the visible book state. By processing the book diff records in sequence, one can reconstruct the full limit order book at any point in time; the order status records add the rich context of rejected orders, order types, and submission parameters that is invisible in the book itself. Many analyses of interest—including those in the companion studies by Albers et al. [1] and Albers et al. [2]—operate directly on the event streams without requiring full book reconstruction. Researchers requiring additional time periods, reconstructed snapshots, or the raw replica command logs—which record every user-submitted action together with the matching engine’s response—are invited to contact the authors. At approximately 880 million order status records per day, the single-month sample contains roughly 27 billion events; the four months of trade data enable longer-horizon investigations where event-level order detail is not required.

⁹<https://doi.org/10.5281/zenodo.18184441>

5. Discussion and Conclusion

We have presented a high-frequency limit order book dataset from Hyperliquid, a large and rapidly growing decentralised exchange, obtained by operating a non-validating blockchain node. The dataset captures the complete lifecycle of every order event—including the vast majority that are rejected by the matching engine—along with counterparty-level trade records and state changes to the limit order book. A sample of this dataset, along with processing tools, is made publicly available to facilitate further research. The dataset derived from Hyperliquid offers significantly higher granularity—particularly including user identifiers and failed order/cancellation attempts—and transparency compared to conventional public datasets like LOBSTER.

The data enables researchers to investigate a range of questions in market microstructure, including for instance:

- **Rejected Orders:** Albers et al. [1] use this dataset to document a massive, previously invisible flow of rejected post-only limit orders and show that a small cohort of traders uses these orders to capture queue priority after spread-widening events.
- **Price Impact (post-event price changes):** Measuring how order flow—including rejected orders—predicts subsequent price movements. Albers et al. [2] use the rejected-order data in this dataset to show that orders which never enter the book are nonetheless significantly correlated with future returns, challenging conventional notions of price impact.
- **Latency Races:** In a latency race, multiple participants compete to act on the same trading opportunity—for instance, several takers racing to hit a mispriced quote, or a market maker racing to cancel before being adversely selected. Conventional data record only the winner; our dataset records the losers as well, in the form of rejected IOC orders (failed attempts to take liquidity) and failed cancellations (stale quotes that were not pulled in time), enabling a far more granular empirical analysis of these competitive scenarios.
- **Trader Populations and Market Ecology:** Because every order carries a pseudonymous wallet identifier, the dataset supports classification of the entire trader population into behavioural archetypes—market makers, directional speculators, arbitrageurs, TWAP executors, and others—and the study of how these populations interact, compete, and evolve over time. Questions that are typically confined to proprietary exchange data, such as concentration of activity among a small number of participants, and the lifecycle of individual trading strategies, become tractable at the population level.

References

- [1] Albers, J., Cucuringu, M., Howison, S., Shestopaloff, A.Y., 2026a. The “neutrinos” of the order book: Pervasive, weakly interacting order flow and its consequences. SSRN Working Paper Available at <https://ssrn.com/abstract=6250738>.
- [2] Albers, J., Cucuringu, M., Howison, S., Shestopaloff, A.Y., 2026b. The price impact of nothing: Rejected orders as predictors of future returns. SSRN Working Paper Available at <https://ssrn.com/abstract=6250378>.
- [3] Cartea, Á., Drissi, F., Monga, M., 2024. Decentralised finance and automated market making: Predictable loss and optimal liquidity provision. *SIAM Journal on Financial Mathematics* 15, 931–959. doi:[10.1137/23M1602103](https://doi.org/10.1137/23M1602103).
- [4] Gould, M.D., Porter, M.A., Williams, S., McDonald, M., Fenn, D.J., Howison, S.D., 2013. Limit order books. *Quantitative Finance* 13, 1709–1742.
- [5] Huang, R., Polak, T., 2011. LOBSTER: Limit order book reconstruction system. SSRN Electronic Journal doi:[10.2139/ssrn.1977207](https://doi.org/10.2139/ssrn.1977207).

- [6] Lehar, A., Parlour, C.A., 2025. Decentralized exchange: The uniswap automated market maker. *The Journal of Finance* 80, 321–374. doi:[10.1111/jofi.13405](https://doi.org/10.1111/jofi.13405).